

# Presenter Guide

Context-Aware Personal AI Assistant: dynamic model routing, persistent memory and retrieval, measured component by component.

---

## 1 · Before you walk in

Setup checklist, 10 minutes

---

## 3 · Numbers to know

The eight figures you will be asked about

---

## 5 · Questions and answers

Seven likely questions

---

## 2 · Who says what

Slide split and timing, 15 minutes

---

## 4 · Live demo script

Five steps, about 4 minutes

---

## 6 · Files and links

What is where

## 1 · BEFORE YOU WALK IN

# Set up 10 minutes early, on Teni's Mac

```
# open Terminal and paste
cd ~/coding/ai-assistant
source .venv/bin/activate
(pgrep -x ollama >/dev/null || ollama serve >/dev/null 2>&1 &)
streamlit run app.py
```

- ☐ **Warm the models.** In *Compare models*, click **Run on all models** once. The first call loads each model (5 to 10 s); after that answers take 1 to 5 s.
- ☐ **Clear old memory.** In *Chat*, click **Wipe memory**.
- ☐ **Reset the weights** to 0.60 / 0.20 / 0.20 (the defaults).
- ☐ **Close Chrome tabs and other apps.** The Mac has 8 GB of RAM.
- ☐ **Open the deck** in a second window, full screen, on slide 1.

## 2 · WHO SAYS WHAT

# Fifteen minutes, three speakers

| Nischay · 3 min   Tejas · 3 min   Teni · 4 min   Demo · 4 min   Q |                             |                                                                                                                                                          |  |  |
|-------------------------------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| Slides                                                            | Speaker                     | What to land                                                                                                                                             |  |  |
| 1–4                                                               | Nischay                     | The problem; our Review 1 claim didn't survive the Perplexity study, so the contribution is now <b>transparent and measured</b> , not "first to combine" |  |  |
| 5–8                                                               | Tejas                       | One pipeline, three switches; the published routing formula; five builds; four benchmarks, all auto-graded                                               |  |  |
| 9–15                                                              | Teni                        | 84.4% honest accuracy; the ablation table; memory-only hallucinates 91.7%; routed 84% vs best single model 78%                                           |  |  |
| 16–20                                                             | Teni drives, Tejas narrates | Live demo (page 4). If anything breaks, stay on slides 16–19: they are real screenshots of the same steps                                                |  |  |
| 21–22                                                             | Nischay                     | Limitations (including over-grounding), next steps for October, questions                                                                                |  |  |

Every slide has speaker notes in the deck. Swap names freely.

### 3 · NUMBERS TO KNOW

## The figures you will be asked about

**84%**

Routed system, 50  
graded tasks

**78%**

Best single model (GPT  
role)

**88%**

Best model per task  
type, in hindsight

**93.3%**

Routing accuracy, 150  
new prompts

**84.4%**

Intent accuracy,  
leakage-free split (was  
reported as 97%)

**96.7%**

Memory recall, 30  
questions, new session

**88.9%**

Retrieval accuracy, 36  
questions

**91.7%**

Hallucination with  
memory but no retrieval

### Ablation: what each component adds

| Build                | Memory recall | RAG accuracy | Hallucination | Says "don't know" |
|----------------------|---------------|--------------|---------------|-------------------|
| Baseline (nothing)   | 0%            | 0%           | 52.8%         | 47.2%             |
| A · routing          | 0%            | 0%           | 13.9%         | 86.1%             |
| B · memory           | 96.7%         | 0%           | 91.7%         | 8.3%              |
| C · retrieval        | 0%            | 88.9%        | 8.3%          | 2.8%              |
| <b>D · all three</b> | <b>96.7%</b>  | <b>88.9%</b> | <b>5.6%</b>   | <b>5.6%</b>       |

### Routing by prompt style

| Style   | Accuracy | Example of a miss                                 |
|---------|----------|---------------------------------------------------|
| Direct  | 100%     | none                                              |
| Verbose | 98%      | long tagline request read as reasoning            |
| Casual  | 82%      | "how many continents are there" read as reasoning |

All five misroutes still went to a strong text model and were answered correctly, so they cost nothing in accuracy.

## Five steps, about four minutes

1

### Chat tab · Model D routing

Type **Write a Python function that reverses a linked list**. Point at the intent bar (coding, 0.99) and the score table: GPT role wins with 0.75.

*"The router shows exactly why it picked this model. Nothing is hidden."*

2

### Move the weights routing

Ask **Write a haiku about monsoon rain**: Claude role. Drag **cost to 1.0** and **accuracy to 0.1**, ask again: it switches to Sonar role. Reset the sliders.

*"The formula is live. Change the priorities and the decision changes."*

3

### Compare models tab all models

Pick **Coding**, click **Run on all models**. Four answers side by side, the router's pick highlighted. Then pick **Search**.

*"Same prompt, every model. This is what the router is choosing between."*

4

### Compare A/B/C/D tab ablation, live

Question **Where do I work?**: only B and D say PayLoop. Then **Who chairs the Helix board?**: only C and D say Meera Iyer.

*"Memory answers questions about you; retrieval answers questions about documents; D does both."*

5

### vs Perplexity tab positioning

Show the table and close the demo.

*"Perplexity is bigger and searches the web. We publish the routing rule and measure every part."*

#### If a model is slow or errors

Move on to the next step, or switch to slides 16–19 (screenshots of the same steps). Don't wait more than 15 seconds.

#### If Model D says "I don't know" to a general question

Expected: it's the over-grounding limitation on slide 21. Say so and move on.

## Seven questions to expect

### Review 2 said 97%. Why is it 84% now?

The old split let paraphrases of test questions into training. Splitting by original question gives 84.4%. The leaky split on today's data gives 100%, which shows the problem.

---

### How is this different from Perplexity?

Perplexity has live web search and much larger models, and we don't claim to beat it there. We publish the routing formula, show why each model is chosen, route five task types, and measure what each component adds. Perplexity publishes none of that.

---

### Why small local models instead of GPT, Claude and Gemini?

Free and reproducible on a laptop. The system is provider-agnostic: switching to the real APIs is a setting in `.env`, planned before the final review.

---

### Why does memory alone hallucinate more than the baseline?

Earlier confident answers stay in the chat history and the model keeps answering confidently instead of admitting it doesn't know. Memory needs retrieval beside it, which is why Model D combines them.

---

### Why did it say "I don't know" to "capital of Australia"?

Retrieval returned the nearest company documents and the "answer only from these" rule made the model refuse. The fix, only applying that rule when passages are relevant, is in next steps.

---

### Is the grading fair?

Code runs against unit tests, maths must hit the exact number, writing is checked against rules like line count or JSON keys. Every answer is logged in the repo so anyone can check.

---

### Routing only beats the best model by 6 points. Is it worth it?

With small models one coder is strong at most text tasks. The gain comes from sending images to the only vision model and keeping it away from maths, where it scores 0%. The spread between big commercial models is wider.

## What is where

| What                            | Where                                                                                             |
|---------------------------------|---------------------------------------------------------------------------------------------------|
| Slides (22, with speaker notes) | "Context-Aware AI Assistant: Review Deck" in Claude. Use its Download menu for PowerPoint or PDF. |
| Report to submit                | BCSE497J_Project_Report_Final.docx and .pdf (25 pages), in this zip and in docs/                  |
| This guide                      | Presenter_Guide.pdf                                                                               |
| Code, data, every model answer  | github.com/teni-chintham/context-aware-ai-assistant (private)                                     |
| Demo screenshots                | docs/screenshots/ in the repo; also in this zip                                                   |
| Raw results                     | results/ablation.json, results/task_bench_raw.jsonl                                               |

### Still to do before the review

- ☐ Get the guide's approval email (was due 01-09)
- ☐ Share the deck from its Share menu
- ☐ Add Nischay and Tejas to the GitHub repo
- ☐ One full rehearsal with the live demo

### For the final review (October)

- ☐ Retrieve only above a relevance threshold
- ☐ More casual training prompts for routing
- ☐ Larger benchmarks, LLM-judge grading
- ☐ Repeat on GPT, Claude and Gemini APIs